

Python Essentials II — Lists, Tuples, Dictionaries & Sets

By the end of today you will understand how Python stores collections of information — not just one value at a time but many values together. You will know when to use a list, tuple, dictionary, or set, and why choosing the right one matters.

Setup reminder: Open VS Code. Create a new file called `day3_practice.py`. Type every example by hand — do not copy-paste. Your fingers need to learn the syntax as much as your brain does.

SECTION 01 Why Do We Need Data Structures?

On Day 2 you learned how to store one piece of information at a time. One name. One age. One price. That works fine for simple programs. But real-world data is never just one thing.

Think about a class register. A teacher does not write one student's name on one page. They write ALL 45 student names together in one list on one page.

A data structure is a way of storing multiple pieces of information together in one organised place, so you can work with all of them efficiently.

Without data structures, storing 45 student names would look like this:

```
student1 = "Rahul"
student2 = "Priya"
student3 = "Arjun"
# ... and so on until student45
# This is terrible. Imagine doing anything useful with this.
```

With a data structure (a list), it looks like this:

```
students = ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]
# All 45 names in one place. Clean. Powerful. Manageable.
```

Python has four main built-in data structures: List, Tuple, Dictionary, and Set. Each one is designed for a different situation. By the end of today you will know exactly which one to reach for and when.

SECTION 02 Lists — Your Most-Used Data Structure

List

What it is: An ordered, changeable collection of items. Items can be of any data type. Duplicates are allowed. Created using square brackets [].

Real-life analogy: Think of a shopping list on paper. It has items in order (milk first, eggs second). You can add new items, cross off items, or change the order. That is a list.

Python example: `fruits = ["apple", "banana", "mango"]`

Creating a list

You can put any type of data inside a list — strings, integers, floats, booleans, even a mix.

```
# A list of student names (strings)
students = ["Rahul", "Priya", "Arjun", "Sneha"]

# A list of scores (integers)
scores = [85, 92, 78, 95, 88]

# A list of prices (floats)
prices = [149.99, 299.50, 89.00]

# A mixed list (allowed, but rarely done in practice)
mixed = ["Rahul", 22, True, 8.5]

# An empty list (you will add items later)
empty_list = []

print(students)      # Output: ['Rahul', 'Priya', 'Arjun', 'Sneha']
print(len(students)) # Output: 4   (len gives you the count)
```

Output:

```
['Rahul', 'Priya', 'Arjun', 'Sneha']
4
```

Accessing items — Indexing

Every item in a list has a position number called an index. Python starts counting from 0, not from 1. This is one of the most important things to understand about Python.

Python uses zero-based indexing. The FIRST item is at index 0, not index 1.

```
students = ["Rahul", "Priya", "Arjun", "Sneha"]
```

```
Index:      0      1      2      3
```

Analogy: Think of a row of flats in a building. In India buildings start from Ground Floor (0), First Floor (1), Second Floor (2). Python thinks the same way.

```
students = ["Rahul", "Priya", "Arjun", "Sneha"]

# Accessing by index
print(students[0])    # Rahul   (first item)
print(students[1])    # Priya   (second item)
print(students[3])    # Sneha   (last item)

# Negative indexing - count from the end
print(students[-1])   # Sneha   (last item)
print(students[-2])   # Arjun   (second from last)
```

Output:

```
Rahul
Priya
Sneha
Sneha
Arjun
```

Negative indexing is very useful when you want the last item but you do not know how long the list is. `students[-1]` always gives the last item, no matter how many students are in the list.

Slicing — Grabbing a portion of a list

Slicing lets you extract a sub-portion of a list. The format is: `list[start : end]` where start is included but end is NOT included.

```
scores = [85, 92, 78, 95, 88, 70, 99]

# Get items from index 1 to index 3 (index 4 not included)
print(scores[1:4])    # [92, 78, 95]

# From the beginning to index 2 (index 3 not included)
print(scores[:3])     # [85, 92, 78]

# From index 4 to the end
print(scores[4:])     # [88, 70, 99]
```

```
# Everything (a copy of the whole list)
print(scores[:])      # [85, 92, 78, 95, 88, 70, 99]

# Last 3 items using negative indexing
print(scores[-3:])    # [88, 70, 99]
```

Output:

```
[92, 78, 95]
[85, 92, 78]
[88, 70, 99]
[85, 92, 78, 95, 88, 70, 99]
[88, 70, 99]
```

Changing, Adding, and Removing Items

Lists are changeable — you can update, add, and remove items after the list is created. These operations are used constantly in real programs.

Changing an existing item

```
students = ["Rahul", "Priya", "Arjun"]

# Rahul's name was entered wrong, fix it
students[0] = "Rahul Sharma"
print(students)  # ['Rahul Sharma', 'Priya', 'Arjun']
```

Output:

```
['Rahul Sharma', 'Priya', 'Arjun']
```

Adding items

```
students = ["Rahul", "Priya", "Arjun"]

# append() — adds to the END of the list
students.append("Sneha")
print(students)  # ['Rahul', 'Priya', 'Arjun', 'Sneha']

# insert() — adds at a SPECIFIC position
students.insert(1, "Mohammed")  # insert at index 1
print(students)  # ['Rahul', 'Mohammed', 'Priya', 'Arjun', 'Sneha']

# extend() — add ALL items from another list
new_students = ["Kavya", "Deepak"]
students.extend(new_students)
print(students)
```

Output:

```
['Rahul', 'Priya', 'Arjun', 'Sneha']  
['Rahul', 'Mohammed', 'Priya', 'Arjun', 'Sneha']  
['Rahul', 'Mohammed', 'Priya', 'Arjun', 'Sneha', 'Kavya', 'Deepak']
```

Removing items

```
fruits = ["apple", "banana", "mango", "banana", "orange"]  
  
# remove() - removes the FIRST occurrence of a value  
fruits.remove("banana")  
print(fruits)      # ['apple', 'mango', 'banana', 'orange']  
  
# pop() - removes and RETURNS the item at an index  
removed = fruits.pop(1)  # removes 'mango' at index 1  
print(removed)      # mango  
print(fruits)      # ['apple', 'banana', 'orange']  
  
# pop() with no argument removes the LAST item  
fruits.pop()  
print(fruits)      # ['apple', 'banana']
```

Output:

```
['apple', 'mango', 'banana', 'orange']  
mango  
['apple', 'banana', 'orange']  
['apple', 'banana']
```

Useful list methods you will use constantly

Method	What it does	Example
len(list)	Count total items	len([1,2,3]) gives 3
.append(x)	Add x to the end	list.append("new")
.insert(i,x)	Add x at position i	list.insert(0, "first")
.remove(x)	Remove first occurrence of x	list.remove("banana")
.pop(i)	Remove and return item at i	list.pop(-1)
.sort()	Sort the list in ascending order	list.sort()
.reverse()	Reverse the order	list.reverse()
x in list	Check if x exists (True/False)	"Rahul" in students
.count(x)	Count how many times x appears	list.count("banana")
.index(x)	Find the index of first x	list.index("mango")

```
scores = [85, 92, 78, 95, 88]

print(len(scores))      # 5
print(max(scores))      # 95 (highest score)
print(min(scores))      # 78 (lowest score)
print(sum(scores))      # 438 (total)

scores.sort()
print(scores)           # [78, 85, 88, 92, 95]

scores.reverse()
print(scores)           # [95, 92, 88, 85, 78]

print(85 in scores)     # True
print(100 in scores)    # False
```

Output:

```
5
95
78
438
[78, 85, 88, 92, 95]
[95, 92, 88, 85, 78]
True
False
```

List Comprehension — A Clean Python Shortcut

List comprehension is a compact way to create a new list by applying an operation to each item of an existing list. It is one of the most loved features in Python.

Without list comprehension (the long way):

```
result = []
for score in scores:
    result.append(score * 2)
```

With list comprehension (the short, Pythonic way):

```
result = [score * 2 for score in scores]
```

Same result. Half the code.

```
scores = [85, 92, 78, 95, 88]

# Double every score
doubled = [score * 2 for score in scores]
print(doubled)      # [170, 184, 156, 190, 176]
```

```
# Get only scores above 85
high_scores = [s for s in scores if s > 85]
print(high_scores)    # [92, 95, 88]

# Convert all names to uppercase
names = ["rahul", "priya", "arjun"]
upper = [name.upper() for name in names]
print(upper)          # ['RAHUL', 'PRIYA', 'ARJUN']
```

Output:

```
[170, 184, 156, 190, 176]
[92, 95, 88]
['RAHUL', 'PRIYA', 'ARJUN']
```

SECTION 03 Tuples — A List That Cannot Be Changed

Tuple

What it is: An ordered, UNCHANGEABLE collection of items. Once created, you cannot add, remove, or modify items. Created using round brackets ().

Real-life analogy: Think of a birthday. It is fixed — you cannot change the day you were born. Or think of GPS coordinates of a city. Bengaluru is always at (12.97, 77.59). These values should never change.

Python example: `birthday = (15, 8, 2002)` or `location = (12.97, 77.59)`

Creating and using tuples

```
# Creating a tuple
birthday      = (15, 8, 2002)      # day, month, year
bengaluru_loc = (12.97, 77.59)     # latitude, longitude
rgb_red       = (255, 0, 0)        # RGB colour values

# Accessing items — same as lists, using index
print(birthday[0])      # 15 (day)
print(birthday[1])      # 8  (month)
print(bengaluru_loc[0]) # 12.97

# Tuples support len, in, slicing — same as lists
print(len(birthday))    # 3
print(15 in birthday)   # True
```

Output:

```
15
8
```

```
12.97
3
True
```

Why would you use a tuple instead of a list?

If tuples are like lists but you cannot change them, why do they exist? There are three good reasons:

- **Safety:** When data must not be changed accidentally, a tuple prevents mistakes. GPS coordinates, date of birth, colour codes.
- **Speed:** Tuples are slightly faster than lists because Python knows they will never change.
- **Dictionary keys:** Tuples can be used as dictionary keys (covered next). Lists cannot.

Rule of thumb: Use a list when the data might change. Use a tuple when the data is fixed and should never be modified. If you are not sure, start with a list.

What happens if you try to change a tuple?

```
birthday = (15, 8, 2002)

# Attempting to change a value
birthday[0] = 20      # This will cause an ERROR
```

TypeError: 'tuple' object does not support item assignment This is intentional. The error is Python protecting your data from accidental changes.

SECTION 04 Dictionaries — Storing Labelled Information

Dictionary

What it is: A collection of key-value pairs. Each item has a label (the key) and a value. Keys must be unique. Created using curly brackets { } with a colon separating each key and value.

Real-life analogy: Think of a real dictionary. You look up a word (the key) and get its meaning (the value). Or think of a student ID card: it has fields like Name, Age, City — each field has a label and a value.

Python example: `student = {"name": "Rahul", "age": 22, "city": "Bengaluru"}`

Creating a dictionary

A dictionary stores information about one thing with multiple labelled attributes. This is how most real-world data is structured.

```
# A student record
student = {
    "name" : "Rahul Sharma",
    "age" : 22,
    "city" : "Bengaluru",
    "gpa" : 8.7,
    "active" : True
}

# A product in an e-commerce store
product = {
    "name" : "Wireless Headphones",
    "price" : 1999.00,
    "brand" : "boAt",
    "in_stock" : True
}
```

Accessing values from a dictionary

You access a value by using its key inside square brackets, or by using the `.get()` method. Never use an index number — dictionaries use keys, not positions.

```
student = {"name": "Rahul", "age": 22, "city": "Bengaluru"}

# Method 1: Square bracket notation
print(student["name"])    # Rahul
print(student["age"])     # 22

# Method 2: .get() -- safer, returns None if key does not exist
print(student.get("city"))    # Bengaluru
print(student.get("email"))    # None (no error, just None)

# With a default value using .get()
print(student.get("email", "Not provided")) # Not provided
```

Output:

```
Rahul
22
Bengaluru
None
Not provided
```

Use `.get()` whenever the key might not exist in the dictionary. If you use square brackets with a missing key, Python throws a `KeyError` and your program crashes. `.get()` is the safe choice.

Adding, updating, and removing entries

```
student = {"name": "Rahul", "age": 22}

# Adding a new key-value pair
student["city"] = "Bengaluru"
print(student)
# {'name': 'Rahul', 'age': 22, 'city': 'Bengaluru'}

# Updating an existing value
student["age"] = 23
print(student["age"])    # 23

# Removing a key-value pair
del student["city"]
print(student)    # {'name': 'Rahul', 'age': 23}

# pop() also removes and returns the value
age = student.pop("age")
print(age)        # 23
print(student)    # {'name': 'Rahul'}
```

Output:

```
{'name': 'Rahul', 'age': 22, 'city': 'Bengaluru'}
23
{'name': 'Rahul', 'age': 23}
23
{'name': 'Rahul'}
```

Looping through a dictionary

In real programs you often need to go through every item in a dictionary. Python gives you three clean ways to do this.

```
student = {"name": "Rahul", "age": 22, "city": "Bengaluru", "gpa": 8.7}

# Loop through all KEYS
for key in student.keys():
    print(key)
# name age city gpa

# Loop through all VALUES
for value in student.values():
    print(value)
# Rahul 22 Bengaluru 8.7

# Loop through both KEY and VALUE together (most useful)
for key, value in student.items():
    print(f"{key}: {value}")
```

Output:

```
name
age
city
gpa
Rahul
22
Bengaluru
8.7
name: Rahul
age: 22
city: Bengaluru
gpa: 8.7
```

A list of dictionaries — how real data looks

In real projects, you will almost never have just one student record. You will have a list of dictionaries — one dictionary per record. This is the most common data pattern you will encounter.

```
# A class register — list of student dictionaries
students = [
    {"name": "Rahul",    "marks": 85, "passed": True},
    {"name": "Priya",    "marks": 92, "passed": True},
    {"name": "Arjun",    "marks": 38, "passed": False},
    {"name": "Sneha",    "marks": 76, "passed": True},
]

# Print each student's result
for student in students:
    status = "PASS" if student["passed"] else "FAIL"
    print(f"{student['name']}: {student['marks']} — {status}")
```

Output:

```
Rahul: 85 — PASS
Priya: 92 — PASS
Arjun: 38 — FAIL
Sneha: 76 — PASS
```

This pattern — a list of dictionaries — is essentially a table of data. Each dictionary is one row. Each key is a column. This is exactly how a CSV file or a database table works. When you load a CSV with Pandas on Day 7, this is the structure underneath.

Set

What it is: An unordered collection where every item is unique. Duplicates are automatically removed. Created using curly brackets {} but WITHOUT key-value pairs.

Real-life analogy: Think of the VIP guest list at an event. Each guest appears only once. If someone tries to add the same name twice, the list still only shows them once. No duplicates, no position order.

Python example: `unique_visitors = {"Rahul", "Priya", "Arjun", "Rahul"}`
— only 3 items stored

Creating and using sets

```
# Creating a set
skills = {"Python", "SQL", "Excel", "Python", "SQL"}
print(skills)      # {'Python', 'SQL', 'Excel'} — duplicates removed!

# Sets are unordered — the order may change each time you print

# Very common use: remove duplicates from a list
scores_with_duplicates = [85, 92, 85, 78, 92, 95, 78]
unique_scores = set(scores_with_duplicates)
print(unique_scores)    # {85, 92, 78, 95}

# Convert back to a sorted list
clean_scores = sorted(list(unique_scores))
print(clean_scores)     # [78, 85, 92, 95]
```

Output:

```
{'Python', 'SQL', 'Excel'}
{85, 92, 78, 95}
[78, 85, 92, 95]
```

Set operations — the really powerful stuff

Sets shine when you need to compare two collections. These operations come directly from mathematics but are incredibly useful in data work.

Imagine two batches of students.

Batch A attended Monday's class. Batch B attended Wednesday's class.

Who attended BOTH? → Intersection

Who attended at least one? → Union

Who attended Monday but NOT Wednesday? → Difference

```
batch_a = {"Rahul", "Priya", "Arjun", "Sneha"}
batch_b = {"Priya", "Mohammed", "Sneha", "Kavya"}
```

```
# Union – all students who attended at least one class
print(batch_a | batch_b)
# {'Rahul', 'Priya', 'Arjun', 'Sneha', 'Mohammed', 'Kavya'}

# Intersection – students who attended BOTH classes
print(batch_a & batch_b)
# {'Priya', 'Sneha'}

# Difference – attended Monday but NOT Wednesday
print(batch_a - batch_b)
# {'Rahul', 'Arjun'}

# Check membership (very fast in sets)
print("Rahul" in batch_a)    # True
print("Kavya" in batch_a)    # False
```

Output:

```
{'Rahul', 'Priya', 'Arjun', 'Sneha', 'Mohammed', 'Kavya'}
{'Priya', 'Sneha'}
{'Rahul', 'Arjun'}
True
False
```

SECTION 06 Choosing the Right Data Structure

This is the most important decision-making skill for today. Read this section carefully and refer back to it whenever you are starting a new program.

Feature	List	Tuple	Dictionary	Set
Ordered?	Yes	Yes	Yes (Py 3.7+)	No
Changeable?	Yes	No	Yes	Yes
Duplicates?	Allowed	Allowed	Keys: No	Not allowed
Syntax	[]	()	{key: val}	{ }
Best for	Any ordered collection	Fixed data (coordinates)	Labelled data (records)	Unique items only

Decision guide — which one should I use?

- Need to store multiple items in order and might add/remove later? → Use a List
- Have fixed data that should never change (coordinates, dates, colour codes)? → Use a Tuple

- Need to store labelled information about one thing (a person, a product, a record)? → Use a Dictionary
- Need to store unique items only, or compare two groups of data? → Use a Set

Quick cheat sheet:

Shopping cart items → List (ordered, can add/remove)

GPS coordinates → Tuple (fixed, should not change)

Student profile → Dictionary (labelled fields)

Unique website visitors → Set (no duplicates needed)

SECTION 07 Interview Prep — Questions to Answer After Today

Practice saying these answers out loud. If you can explain it simply, you understand it well.

Q — What is the difference between a list and a tuple?

Both are ordered collections of items in Python. The key difference is mutability. A list is mutable — you can add, remove, and change items after creation. A tuple is immutable — once created, it cannot be changed. Lists use square brackets, tuples use round brackets. Use a list when data might change, use a tuple for fixed data like coordinates or dates that should stay constant.

Q — What is a dictionary and when would you use one?

A dictionary stores key-value pairs — each piece of data has a label. It is like a real dictionary where you look up a word (key) and get its meaning (value). You would use a dictionary when you need to store multiple attributes about one thing. For example, a student record with name, age, GPA, and city. Dictionaries are also very fast for looking up values by key because Python uses a technique called hashing internally.

Q — What is a set and what makes it different from a list?

A set stores unique items only and is unordered. Unlike a list, sets automatically remove duplicates. If you add the same item twice, only one copy is kept. Sets are extremely useful for two things: removing duplicates from a collection, and performing mathematical operations like union, intersection, and difference between two groups of data. The main trade-off is that sets are unordered, so you cannot access items by index.

Q — What is list comprehension? Can you give an example?

List comprehension is a compact, readable way to create a new list by applying an expression to each item in an existing collection. For example, if I have a list of prices and want a new list with 10% discount applied, I can write: `discounted = [price * 0.9 for price in prices]`. This is equivalent to writing a for loop that appends to a new list, but it is shorter and considered more Pythonic. You can also add conditions: `high_scores = [s for s in scores if s > 80]`.

Term	Definition
List	Ordered, changeable collection in square brackets. Duplicates allowed.
Tuple	Ordered, UNCHANGEABLE collection in round brackets. Use for fixed data.
Dictionary	Key-value pairs in curly brackets. Access values by their label (key).
Set	Unordered collection of UNIQUE items. Duplicates removed automatically.
Index	The position number of an item in a list or tuple. Starts at 0.
Slicing	Extracting a portion of a list using [start:end] notation.
Key	The label in a dictionary used to look up a value.
Value	The data stored against a key in a dictionary.
Mutable	Can be changed after creation. Lists and dicts are mutable.
Immutable	Cannot be changed after creation. Tuples and strings are immutable.
.append()	Adds one item to the end of a list.
.get()	Safely access a dictionary value. Returns None if key not found.
List comprehension	A short, clean way to create a list: [expr for item in list]

SECTION HW Homework — Submit Before Day 4

Create a file called `day3_homework.py` and push it to your GitHub repo before midnight. This is your daily pass/fail submission.

Exercise 1 — Student Mark Sheet (Beginner)

Create a list of 5 student names and a separate list of their marks. Print each student name along with their mark using their index. Then print the highest mark, lowest mark, and class average.

Hint: use `max()`, `min()`, and `sum()` with `len()` for the average.

Exercise 2 — Remove Duplicates (Beginner-Intermediate)

Create a list of city names that has some duplicates in it. Use a set to remove all duplicates, then convert back to a sorted list and print it.

Hint: `sorted(list(set(your_list)))` does it in one line.

Exercise 3 — Student Profile (Intermediate)

Ask the user to enter: their name, age, city, and favourite subject. Store all four pieces of information in a dictionary. Then print a formatted profile card using f-strings that shows all four fields neatly.

Exercise 4 — Class Register (Intermediate)

Create a list of at least 4 student dictionaries. Each dictionary should have: name, marks, and passed (True if marks ≥ 40 , False otherwise). Loop through the list and print each student's name and whether they passed or failed.

Exercise 5 — List Comprehension Practice (Intermediate)

Given a list of 8 numbers, use list comprehension to: (1) create a new list with every number squared, (2) create a new list with only the even numbers, (3) create a new list with only numbers greater than 50.

Hint for even numbers: a number is even if `number % 2 == 0`

Stretch Challenge — Tier A students

Create a simple inventory management system using a list of dictionaries. Each item should have: product name, price, and quantity. Then write code to: (1) print all products under Rs. 500 using list comprehension, (2) calculate the total inventory value (price x quantity for each item, then sum), (3) find the most expensive product.